

Lab 2: Introduction to LabVIEW and USRP

Objective: The main objective of this lab is to get familiar with LabVIEW graphical programming basics, programming tools and debugging tools.

1. Data Types and Structures

Like other programming languages, NI LabVIEW software has many different data types, and ways to organize those data types. They can be distinguished by color and decoration, and they are each used for particular reasons. This section illustrates the capabilities of various data types and structures, and when to use one over another.

This section includes:

- String Data Type
- Numeric Data Type
- Boolean Data Type
- Dynamic Data Type
- Data Structures in LabVIEW
- Creating Array Controls and Indicators
- Using Arrays With Loops
- Clusters
- Creating Clusters
- Using Cluster Functions
- Enums

String Data Type

A string is a sequence of displayable or nondisplayable ASCII characters. Strings provide a platform-independent format for information and data.



Some of the more common applications of strings include the following:

- Creating simple text messages.
- Controlling instruments by sending text commands to the instrument and returning data values in the form of either ASCII or binary strings, which you then convert to numeric values.
- Storing numeric data to disk. To store numeric data in an ASCII file, you must first convert numeric data to strings before writing the data to a disk file.
- Instructing or prompting the user with dialog boxes.

On the front panel, strings appear as tables, text entry boxes, and labels. LabVIEW includes built-in VIs and functions you can use to manipulate strings, including formatting strings, parsing strings, and other editing. LabVIEW represents string data with the color pink.

Numeric Data Type

LabVIEW represents numeric data as floating-point numbers, fixed-point numbers, integers, unsigned integers, and complex numbers. Double and Single precision as well as Complex numeric data is represented with the color orange in LabVIEW. All Integer numeric data is represented with the color blue.



Note: The difference among the numeric data types is the number of bits they use to store data and the data values they represent.

Boolean Data Type

LabVIEW stores Boolean data as 8-bit values. A Boolean can be used in LabVIEW to represent a 0 or 1, or a TRUE or FALSE. If the 8-bit value is zero, the Boolean value is FALSE. Any nonzero value represents TRUE.



Common applications for Boolean data include representing digital data and serving as a front panel control that acts as a switch that has a mechanical action often used to control an execution structure such as a Case structure. A Boolean control is typically used as the conditional statement to exit a While Loop. In LabVIEW, the color green represents Boolean data.

Dynamic Data Type

Most Express VIs accept and/or return the dynamic data type, which appears as a dark blue terminal.



Using the Convert to Dynamic Data and Convert from Dynamic Data VIs, you can convert floating-point numeric or Boolean data of the following data types :

- 1D array of waveforms
- 1D array of scalars
- 1D array of scalars – most recent value
- 1D array of scalars – single channel
- 2D array of scalars – columns are channels
- 2D array of scalars – rows are channels
- Single scalar
- Single waveform

Wire the dynamic data type to an indicator that can best present the data. Indicators include a graph, chart, or numeric, or Boolean indicator. However, because dynamic data undergoes an automatic

conversion to match the indicator to which it is wired, Express VIs can slow down the block diagram execution speed.

The dynamic data type is for use with Express VIs. Most other VIs and functions that are shipped with LabVIEW do not accept this data type. To use a built-in VI or function to analyze or process the data the dynamic data type includes, you must convert the dynamic data type.

Data Structures in LabVIEW

Sometimes it is beneficial to group data related to one another. Use arrays and clusters to group related data in LabVIEW. Arrays combine data of the same data type into one data structure, and clusters combine data of multiple data types into one data structure.

Arrays

An array consists of elements and dimensions. Elements are the data that make up the array.

A dimension is the length, height, or depth of an array. An array can have one or more dimensions and as many as $(2^{31}) - 1$ elements per dimension, memory permitting.

You can build arrays of numeric, Boolean, path, string, waveform, and cluster data types. Consider using arrays when you work with a collection of similar data and when you perform repetitive computations. Arrays are ideal for storing data you collect from waveforms or data generated in loops, where each iteration of a loop produces one element of the array.

Note: Array indexes in LabVIEW are zero-based. The index of the first element in the array, regardless of its dimension, is zero.

Array elements are ordered. An array uses an index so you can readily access any particular element. The index is zero-based, which means it is in the range 0 to $n - 1$, where n is the number of elements in the array. For example, $n = 12$ for the twelve months of the year, so the index ranges from 0 to 11. March is the third month, so it has an index of 2.

Figure 1 shows an example of an array of numerics. The first element shown in the array (3.00) is at index 1, and the second element (1.00) is at index 2. The element at index 0 is not shown in this image because element 1 is selected in the index display. The element selected in the index display always refers to the element shown in the upper left corner of the element display.

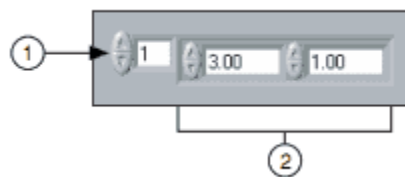


Figure 1. Array Index Display, Numeric Control Array

Creating Array Controls and Indicators

Create an array control or indicator on the front panel by adding an array shell to the front panel, as shown in Figure 2, and dragging a data object or element, such as a numeric or string control, into the array shell.

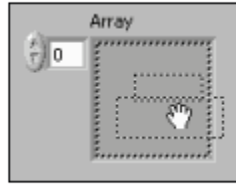


Figure 2. Placing a Numeric Control in an Array Shell

If you attempt to drag an invalid control or indicator into the array shell, you are unable to place the control or indicator in the array shell.

You must insert an object in the array shell before you use the array on the block diagram. Otherwise, the array terminal appears black with an empty bracket and has no data type associated with it.

2D Arrays

The previous examples use 1D arrays. A 2D array stores elements in a grid. It requires a column index and a row index to locate an element, both of which are zero-based. Figure 3 shows an eight-column by eight-row 2D array which contains $8 \times 8 = 64$ elements.

	Column Index							
	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								
6								
7								

Figure 3. 2D Array

To add a multidimensional array to the front panel, right-click the index display and select **Add Dimension** from the shortcut menu. You also can resize the index display until you have as many dimensions as you want.

Initializing Arrays

You can initialize an array or leave it uninitialized. When an array is initialized, you defined the number of elements in each dimension and the contents of each element. An uninitialized array contains a fixed number of dimensions but no elements. Figure 4 shows an uninitialized 2D array control. Notice that the elements are all dimmed. This indicates that the array is uninitialized.

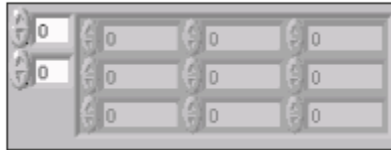


Figure 4. 2D Uninitialized Array

In Figure 5, six elements are initialized. In a 2D array, after you initialize an element in a row, the remaining elements in that row are initialized and populated with the default value for the data type. For example, in Figure 5, if you enter 4 into the element in the first column, third row, the elements in the second and third column in the third row are automatically populated with a 0.

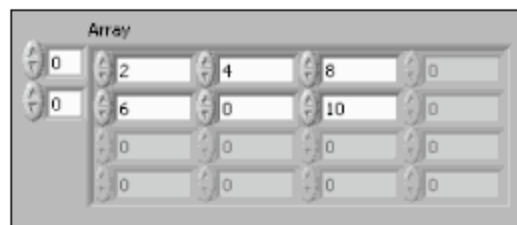


Figure 5. An Initialized 2D Array With Six Elements

Creating Array Constants

To create an array constant on the block diagram, select an **Array Constant** on the Functions palette, place the array shell on the block diagram, and place a string constant, numeric constant, Boolean constant, or cluster constant in the array shell. You can use an array constant to store constant data or as a basis for comparison with another array.

Using Arrays with Loops

Auto-Indexing Array Inputs

If you wire an array to or from a For Loop or While Loop, you can link each iteration of the loop to an element in that array by enabling auto-indexing. The tunnel image changes from a solid square to the image to indicate auto-indexing. Right-click the tunnel and select **Enable Indexing** or **Disable Indexing** from the shortcut menu to toggle the state of the tunnel.

Array Inputs

If you enable auto-indexing on an array wired to a For Loop input terminal, LabVIEW sets the count terminal to the array size so you do not need to wire the count terminal. Because you can use For Loops to process arrays one element at a time, LabVIEW enables auto-indexing by default for every array you wire to a For Loop. You can disable auto-indexing if you do not need to process arrays one element at a time.

In Figure 6, the For Loop executes a number of times equal to the number of elements in the array. Normally, if the **Loop Count** terminal of the For Loop is not wired, the run arrow is broken. However, in this case the **Run Arrow** button is not broken.

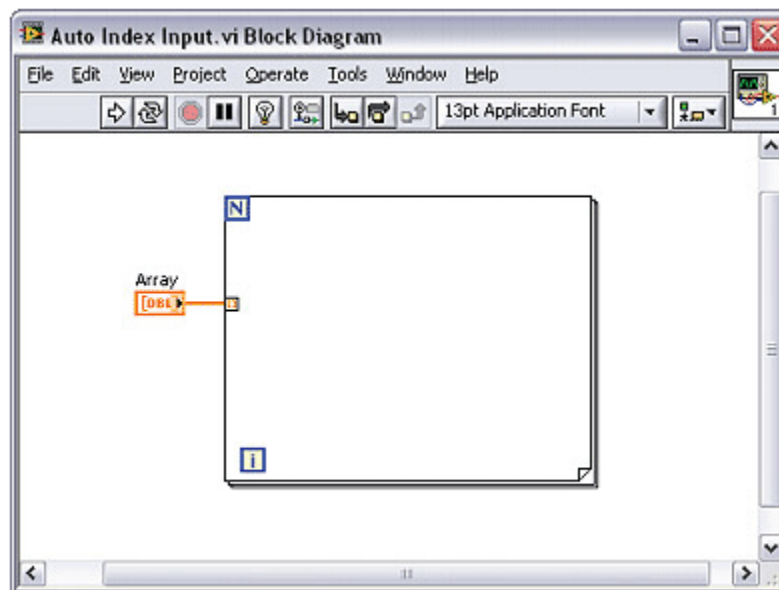


Figure 6. Array Used to Set For Loop Count

If you enable auto-indexing for more than one tunnel or if you wire the count terminal, the actual number of iterations becomes the smaller of the choices. For example, if two auto-indexed arrays enter the loop, with 10 and 20 elements respectively, and you wire a value of 15 to the count terminal, the loop still executes only 10 times, indexing all elements of the first array but only the first 10 elements of the second array.

Array Outputs

When you auto-index an array output tunnel, the output array receives a new element from every iteration of the loop. Therefore, auto-indexed output arrays are always equal in size to the number of iterations. The wire from the output tunnel to the array indicator becomes thicker as it changes to an array at the loop border, and the output tunnel contains square brackets representing an array.



Figure 7. Auto-Indexed Output

Right-click the tunnel at the loop border and select **Enable Indexing** or **Disable Indexing** from the shortcut menu to enable or disable auto-indexing. Auto-indexing for While Loops is disabled by default. For example, disable auto-indexing if you need only the last value passed out of the tunnel.

Creating 2D Arrays

You can use two For Loops, nested one inside the other, to create a 2D array. The outer For Loop creates the row elements, and the inner For Loop creates the column elements.

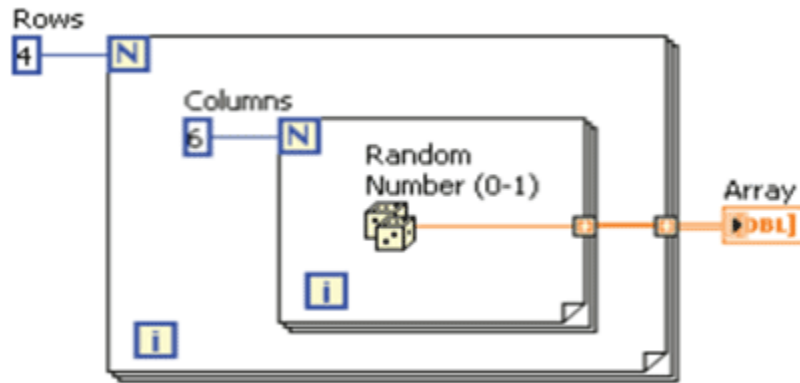


Figure 8. Creating a 2D Array

Clusters

Clusters group data elements of mixed types. An example of a cluster is the LabVIEW error cluster, which combines a Boolean value, a numeric value, and a string. A cluster is similar to a record or a struct in text-based programming languages.

Bundling several data elements into clusters eliminates wire clutter on the block diagram and reduces the number of connector pane terminals that subVIs need. The connector pane has, at most, 28 terminals. If your front panel contains more than 28 controls and indicators that you want to pass to another VI, group some of them into a cluster and assign the cluster to a terminal on the connector pane.

Most clusters on the block diagram have a pink wire pattern and data type terminal. Error clusters have a dark yellow wire pattern and data type terminal. Clusters of numeric values, sometimes referred to as points, have a brown wire pattern and data type terminal. You can wire brown numeric clusters to Numeric functions, such as Add or Square Root, to perform the same operation simultaneously on all elements of the cluster.

Order of Cluster Elements

Although cluster and array elements are both ordered, you must unbundle all cluster elements at once using the Unbundle function. You can use the Unbundle by Name function to unbundle cluster elements by name. If you use the Unbundle by Name function, each cluster element must have a label. Clusters also differ from arrays in that they are a fixed size. Like an array, a cluster is either a control or an indicator. A cluster cannot contain a mixture of controls and indicators.

Bundling several data elements into clusters eliminates wire clutter on the block diagram and reduces

the number of connector pane terminals that subVIs need. The connector pane has, at most, 28 terminals. If your front panel contains more than 28 controls and indicators that you want to pass to another VI, group some of them into a cluster and assign the cluster to a terminal on the connector pane.

Most clusters on the block diagram have a pink wire pattern and data type terminal. Error clusters have a dark yellow wire pattern and data type terminal. Clusters of numeric values, sometimes referred to as points, have a brown wire pattern and data type terminal. You can wire brown numeric clusters to Numeric functions, such as Add or Square Root, to perform the same operation simultaneously on all elements of the cluster.

Creating Clusters

Controls and Indicators

Create a cluster control or indicator on the front panel by adding a cluster shell to the front panel, as shown in Figure 9, and dragging a data object or element, which can be a numeric, Boolean, string, path, refnum, array, or cluster control or indicator, into the cluster shell. Resize the cluster shell by dragging the cursor while you place the cluster shell.

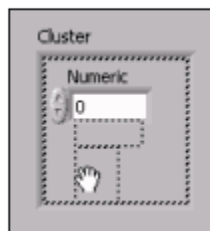


Figure 9. Creation of a Cluster Control

Figure 10 is an example of a cluster containing three controls: a string, a Boolean switch, and a numeric. A cluster is either a control or an indicator; it cannot contain a mixture of controls and indicators.

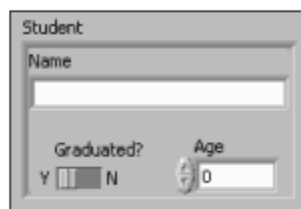


Figure 10. Cluster Control Example

Creating Cluster Constants

To create a cluster constant on the block diagram, select a **Cluster Constant** on the Functions palette, place the cluster shell on the block diagram, and place a string constant, numeric constant, Boolean constant, or cluster constant in the cluster shell. You can use a cluster constant to store constant data or

as a basis for comparison with another cluster.

If you have a cluster control or indicator on the front panel window and you want to create a cluster constant containing the same elements on the block diagram, you can either drag that cluster from the front panel window to the block diagram or right-click the cluster on the front panel window and select **Create»Constant** from the shortcut menu.

Using Cluster Functions

Use the Cluster functions to create and manipulate clusters. For example, you can perform tasks similar to the following:

- Extract individual data elements from a cluster.
- Add individual data elements to a cluster.
- Break a cluster into its individual data elements.

Use the Bundle function to assemble a cluster, use the Bundle function and Bundle by Name function to modify a cluster, and use the Unbundle function and the Unbundle by Name function to disassemble clusters.

You also can place the Bundle, Bundle by Name, Unbundle, and Unbundle by Name functions on the block diagram by right-clicking a cluster terminal on the block diagram and selecting **Cluster, Class & Variant Palette** from the shortcut menu. The Bundle and Unbundle functions automatically contain the correct number of terminals. The Bundle by Name and Unbundle by Name functions appear with the first element in the cluster. Use the Positioning tool to resize the Bundle by Name and Unbundle by Name functions to show the other elements of the cluster.

Assembling Clusters

Use the Bundle function to assemble a cluster from individual elements or to change the values of individual elements in an existing cluster without having to specify new values for all elements. Use the Positioning tool to resize the function or right-click an element input and select Add Input from the shortcut menu.

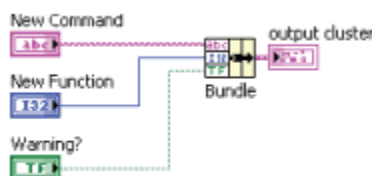


Figure 11. Assembling a Cluster on the Block Diagram

Modifying a Cluster

If you wire the cluster input, you can wire only the elements you want to change. For example, the Input Cluster shown in Figure 12 contains three controls.

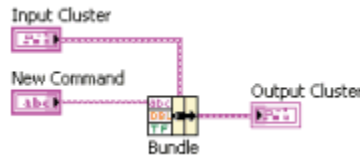


Figure 12. Bundle Used to Modify a Cluster

If you know the cluster order, you can use the Bundle function to change the Command value by wiring the elements shown in **Figure 12**.

You can also use the Bundle by Name function to replace or access labeled elements of a cluster. The Bundle by Name function works like the Bundle function, but instead of referencing cluster elements by their cluster order, it references them by their owned labels. You can access only elements with owned labels. The number of inputs does not need to match the number of elements in an output cluster.

Use the Operating tool to click an input terminal and select an element from the pull-down menu. You also can right-click the input and select the element from the Select Item shortcut menu.

In Figure 13, you can use the Bundle by Name function to update the values of Command and Function with the values of New Command and New Function.

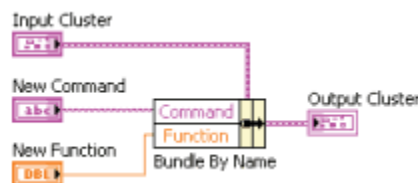


Figure 13. Bundle by Name Used to Modify a Cluster

Use the Bundle by Name function for data structures that might change during development. If you add a new element to the cluster or modify its order, you do not need to rewire the Bundle by Name function because the names are still valid.

Disassembling Clusters

Use the Unbundle function to split a cluster into its individual elements. Use the Unbundle by Name function to return the cluster elements whose names you specify. The number of output terminals does not depend on the number of elements in the input cluster. Use the Operating tool to click an output terminal and select an element from the pull-down menu. You also can right-click the output terminal and select the element from the Select Item shortcut menu.

For example, if you use the Unbundle function with the cluster in Figure 14, it has four output terminals that correspond to the four controls in the cluster. You must know the cluster order so you can associate the correct Boolean terminal of the unbundled cluster with the corresponding switch in the cluster. In this example, the elements are ordered from top to bottom starting with element 0. If you use the Unbundle by Name function, you can have an arbitrary number of output terminals and access individual elements by name in any order.

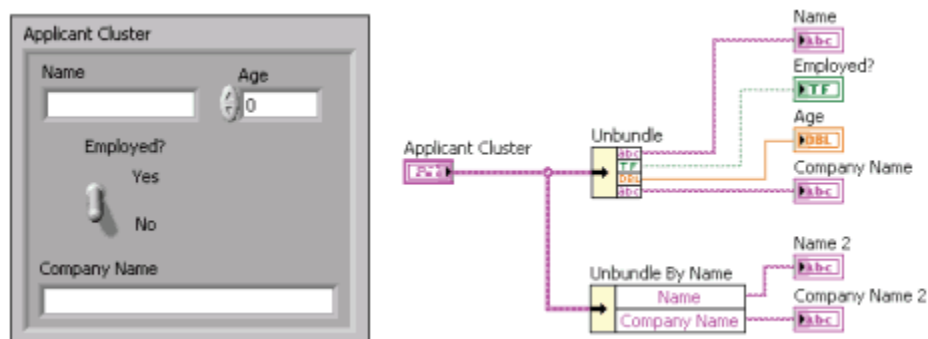


Figure 14. Unbundle and Unbundle By Name

Enums

An enum (enumerated control, constant, or indicator) is a combination of data types. An enum represents a pair of values, a string and a numeric, where the enum can be one of a list of values. For example, if you created an enum type called Month, the possible value pairs for a Month variable are January-0, February-1, and so on through December-11.

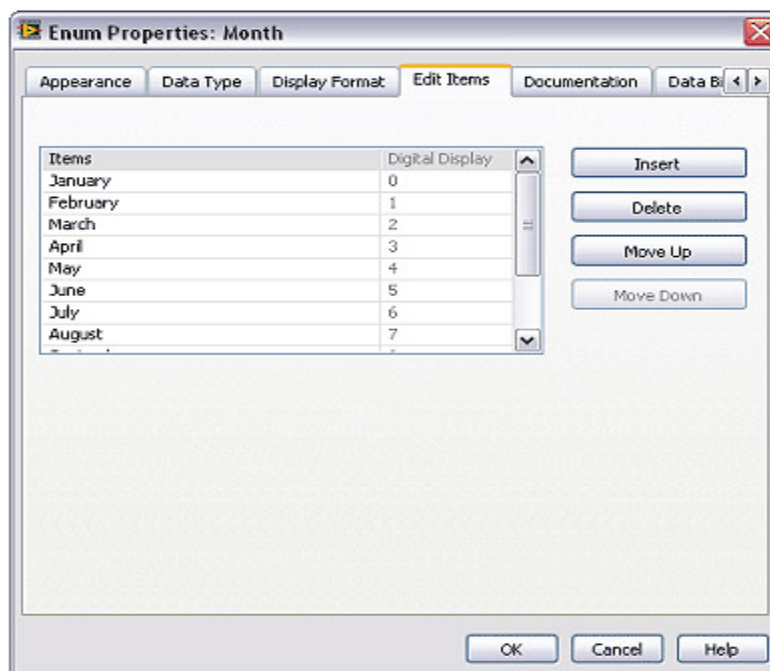


Figure 15. Properties for the Month Enumerated Control

Figure 15 shows an example of these data pairs in the Properties dialog box for an enumerated control. This is directly accessed by right-clicking the enum control and selecting **Edit Items**.

Enums are useful because it is easier to manipulate numbers on the block diagram than strings. Figure 16 shows the Month enumerated control, the selection of a data pair in the enumerated control, and the corresponding block diagram terminal.

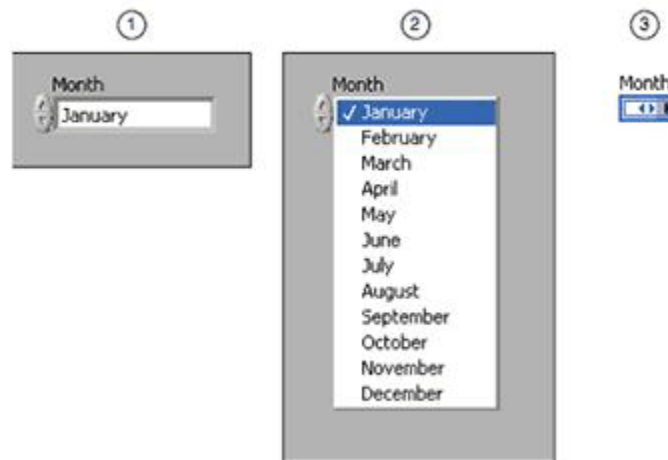


Figure 16. Month Enumerated Control, Selection in Control, Block Diagram Terminal

2. Graphical Programming

Data Flow

LabVIEW follows a dataflow model for running VIs. A block diagram node executes when it receives all required inputs. When a node executes, it produces output data and passes the data to the next node in the dataflow path. The movement of data through the nodes determines the execution order of the VIs and functions on the block diagram.

Visual Basic, C++, JAVA, and most other text-based programming languages follow a control flow model of program execution. In control flow, the sequential order of program elements determines the execution order of a program.

For a dataflow programming example, consider a block diagram that adds two numbers and then subtracts 50.00 from the result of the addition, as shown in Figure 1. In this case, the block diagram executes from left to right, not because the objects are placed in that order, but because the Subtract function cannot execute until the Add function finishes executing and passes the data to the Subtract function. Remember that a node executes only when data is available at all of its input terminals and supplies data to the output terminals only when the node finishes execution.

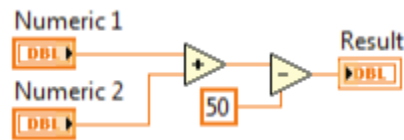


Figure 1. Dataflow Programming Example

In Figure 2, consider which code segment would execute first – the Add, Random Number, or Divide function. You cannot know because inputs to the Add and Divide functions are available at the same time, and the Random Number function has no inputs. In a situation where one code segment must execute before another, and no data dependency exists between the functions, use other programming methods, such as sequence structures or error clusters, to force the order of execution.

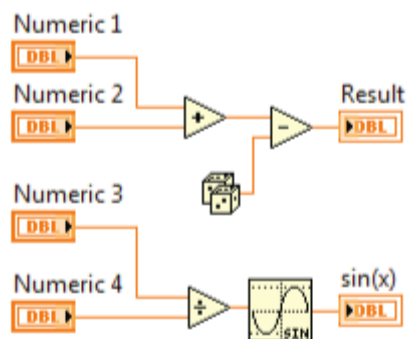


Figure 2. Dataflow Example for Multiple Code Segments

Wires

You transfer data among block diagram objects through wires. In figures 1 and 2, wires connect the control and indicator terminals to the Add and Subtract function. Each wire has a single data source, but you can wire it to many VIs and functions that read the data. Wires are different colors, styles, and thicknesses, depending on their data types.



Figure 3. Broken Wire Example

A broken wire appears as a dashed black line with a red X in the middle, as shown in Figure 3. Broken wires occur for a variety of reasons, such as when you try to wire two objects with incompatible data types. Table 1 shows the most common wire types.

Data Type	Scalar	1D Array	2D Array	Color
Numeric - Floating Point				Orange
Numeric - Integer				Blue
Boolean				Green
String				Pink

Table 1. Common Wire Types

3. Introduction to NI USRP-2922

Overview

The NI USRP-2922 software defined radio (SDR) transceiver is an affordable and easy-to-use RF platform for rapidly prototyping applications in wireless communications research and education. It has continuous frequency coverage from 400 MHz to 4.4 GHz that includes some of the most commonly researched technologies such as cellular, GNSS, 2.4 GHz WiFi, L band radar, and S band radar. This SDR device allows simultaneous transmit and receive with up to 40 MHz of instantaneous bandwidth. The integrated multiple input, multiple output (MIMO) and synchronization port enables plug-and-play 2x2 MIMO. The external reference clock inputs allow the synchronization of up to eight devices for 8x8 MIMO or higher-channel-count direction-finding applications.

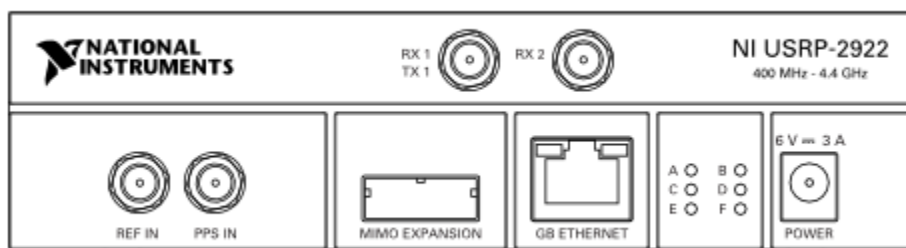


Figure 8. Front Panel

NI USRP-2922 Kit Contents

- NI USRP-2922 device with power supply
- Modulation Toolkit
- LabVIEW Digital Filter Design Toolkit

- LabVIEW MathScript RT Module
- 3 m Ethernet cable
- 0.3 m SMA-SMA loopback cable
- 30 dB SMA inline attenuator

USRP Hardware:

The NI USRP connects to a host PC to act as a software-defined radio. Incoming signals attached to the standard SMA connector are mixed down using a direct-conversion receiver (DCR) to baseband I/Q components, which are sampled by a 2-channel, 100 MS/s, 14-bit analog-to-digital converter (ADC). The digitized I/Q data follows parallel paths through a digital downconversion (DDC) process that mixes, filters, and decimates the input 100 MS/s signal to a user-specified rate. The downconverted samples, when represented as 32-bit numbers (16 bits each for I and Q), are passed to the host computer at up to 20 MS/s over a standard Gigabit Ethernet connection.

For transmission, baseband I/Q signal samples are synthesized by the host computer and fed to the USRP at up to 20 MS/s over Gigabit Ethernet when represented with 32-bits (16-bits each for the I and Q components). The USRP hardware interpolates the incoming signal to 400 MS/s using a digital upconversion (DUC) process and then converts the signal to analog with a dual-channel, 16-bit digital-to-analog converter (DAC). The resulting analog signal is then mixed up to the specified carrier frequency. An available 8-bit mode, in which 16-bits total are used to represent the I and Q values of a downconverted sample or sample to be upconverted, can enable a transfer rate of up to 40 MS/s over the Gigabit Ethernet connection between the host PC and the USRP. USRP block diagram is shown in figure 9.

Software Define Radio (SDR):

The Wireless Innovation Forum defines Software Defined Radio (SDR) as:

“Radio in which some or all of the physical layer functions are software defined.”

SDR refers to the technology wherein software modules running on a generic hardware platform are used to implement radio functions. Combine the NI USRP hardware with LabVIEW software for the flexibility and functionality to deliver a platform for rapid prototyping involving physical layer design, wireless signal record & playback, signal intelligence, algorithm validation, and more.

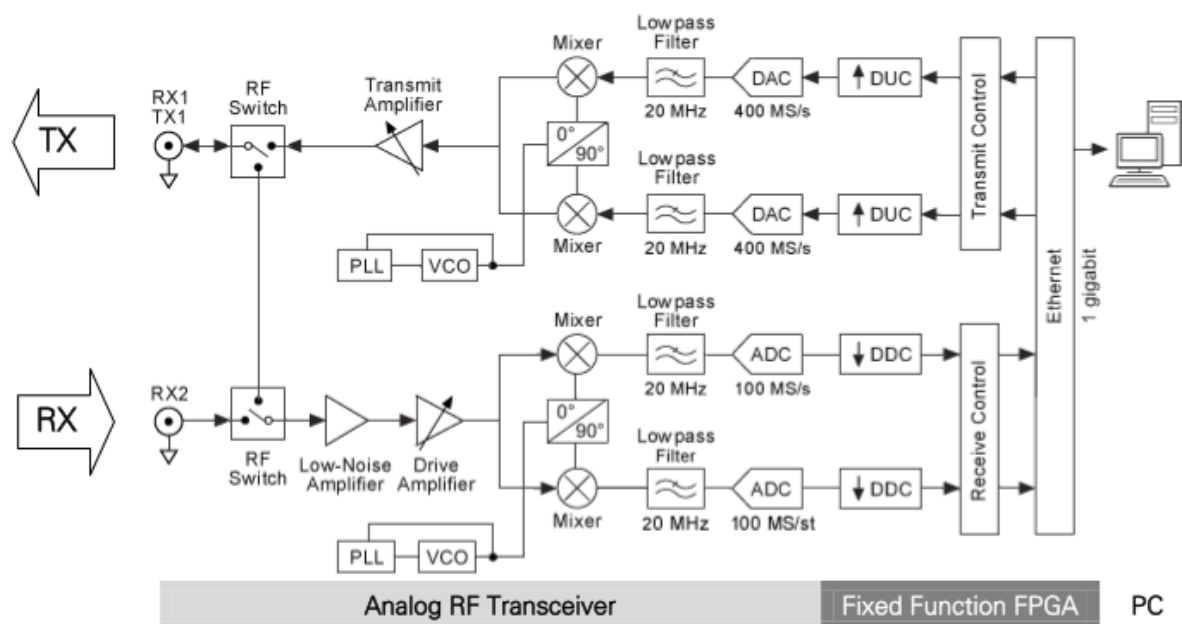


Figure 9. USRP Block Diagram

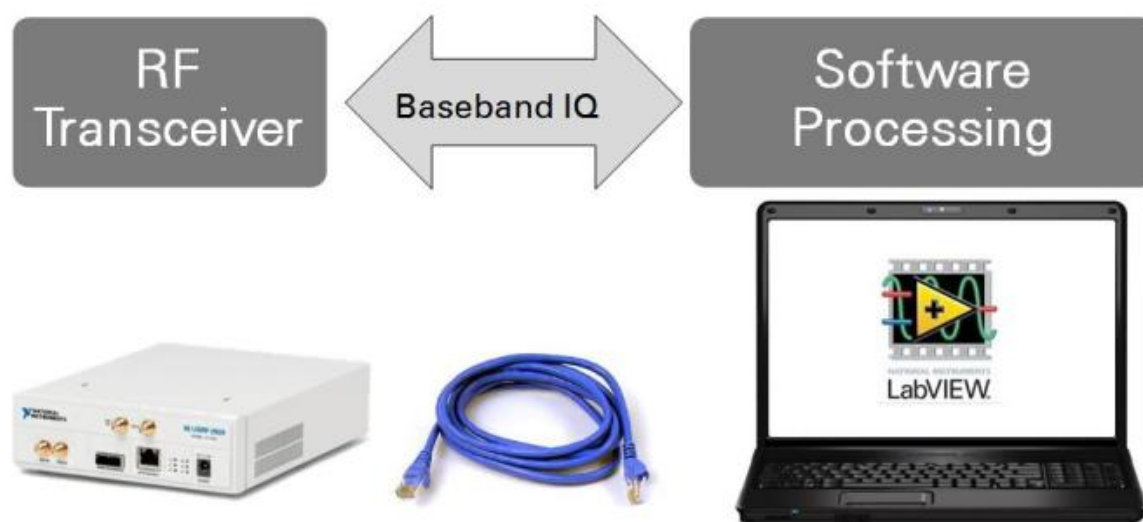


Figure 10. Simplified Overview of a SDR Setup Built Around an NI USRP

Role of Computer in SDR:

A software-defined radio system is a radio communication system in which certain hardware components are implemented in software. These hardware components include filters, amplifiers, modulators, and demodulators. Because these components are defined in software, you can adjust a

software-defined radio system as needed without making significant hardware changes. Since computers today may contain very fast processors and high-speed interfaces, we can leverage these abilities for our software defined radio by implementing them on a computer quickly, using LabVIEW.

Lab Task

In-Lab Task 1:

Create a VI in while loop. Create the logic to find out which (the first) index in the array that have the number greater than 30. See front panel below.



In-Lab Task 2:

Take array as input then implement the following operation on it both using Mathscript node and LabVIEW functions:

- Create array X then take square of array X and name it Y.
- Extract first elements of array X.
- Find the dot product of X and Y and name it Z.

In-Lab Task 3:

Make an array $v = [1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9\ 10]$, develop an algorithm such that the first element of the vector is multiplied by $\text{length}(v)$, second element by $\text{length}(v)-1$ and similarly the last element i.e. 10 is multiplied by $\text{length}(v)-9$. The final output array should be $f = [10\ 18\ 24\ 28\ 30\ 30\ 28\ 24\ 18\ 10]$. The algorithm devised should only use the array size function of array v to achieve array f .

Post-Lab Task 1: Answer the following:

- What is frequency range of USRP kit 2922?
- How many ports are available for transmission and reception on USRP kit 2922?
- What are hardware components which are performed in computer/LabVIEW in SDR?

Post-Lab Task 2:

Create a VI in for loop. Create logic to find out if a number in array is greater than 10. See front panel in figure below.

